

Flexible Glass

Liquid Glass for Unity uGUI

User manual

Flexible Glass provides two glass components for Unity uGUI:

- **UIGlass** merges panels and subtracts shapes to create holes and cutouts.
- **GlassImage** is a uGUI Image with glass rendering, Canvas ordering, and UI masking.

Both support parametric shapes and sprite outlines. Add **FlexibleGlassFeature** to the camera's renderer to supply their background capture, blur, and lighting. The feature has built-in blur and can also use Flexible Blur.

Contents

Getting started	2
Choosing a component	4
Shapes	5
Combining panels	7
Surface and refraction	9
Lighting and shadows	10
Blur and renderer settings	11
Cameras and layered UI	13
Raycasting and scripting	17
Performance and retained fields	18
Troubleshooting	19

Before you begin

Use a Universal Render Pipeline (URP) project with uGUI, compute shaders, and shader model 4.5 support. The Built-in Render Pipeline and HDRP are not supported. Check these requirements on the target device as well as in the Editor.

Web builds require WebGPU; WebGL 2 cannot run the compute shaders used by Flexible Glass.

Support: hurleybird@gmail.com.

Getting started

Add the renderer feature

1. Import Flexible Glass into a URP project and allow shaders to finish importing.
2. Find the URP Renderer Data asset used by your camera. Check the camera's Renderer selection, and the URP asset assigned to the active Quality level if it differs from Graphics settings.
3. On that Renderer Data asset, choose Add Renderer Feature > Flexible Glass Feature.
4. Choose the Render Pass Event for your Canvas arrangement. The Cameras and layered UI chapter explains the ordering, including when two features are needed.

Add glass to a Canvas

Use `GameObject > UI > UI Glass` for joining and subtracting shapes, or `GameObject > UI > Glass Image` for an Image component. In Unity 6.3 and later, these commands are under `UI (Canvas)`.

In Glass Source, assign the camera whose image should appear through the glass. That camera's renderer must contain `FlexibleGlassFeature`. Feature # selects among Glass features on that renderer, starting at 0; it does not count other kinds of renderer feature. The selector is hidden when there is only one Glass feature.

Both components need a parent Canvas. `GlassImage` is drawn by that Canvas; `UIGlass` is drawn by the renderer feature. This difference determines which UI can appear behind the glass.

Finding the controls

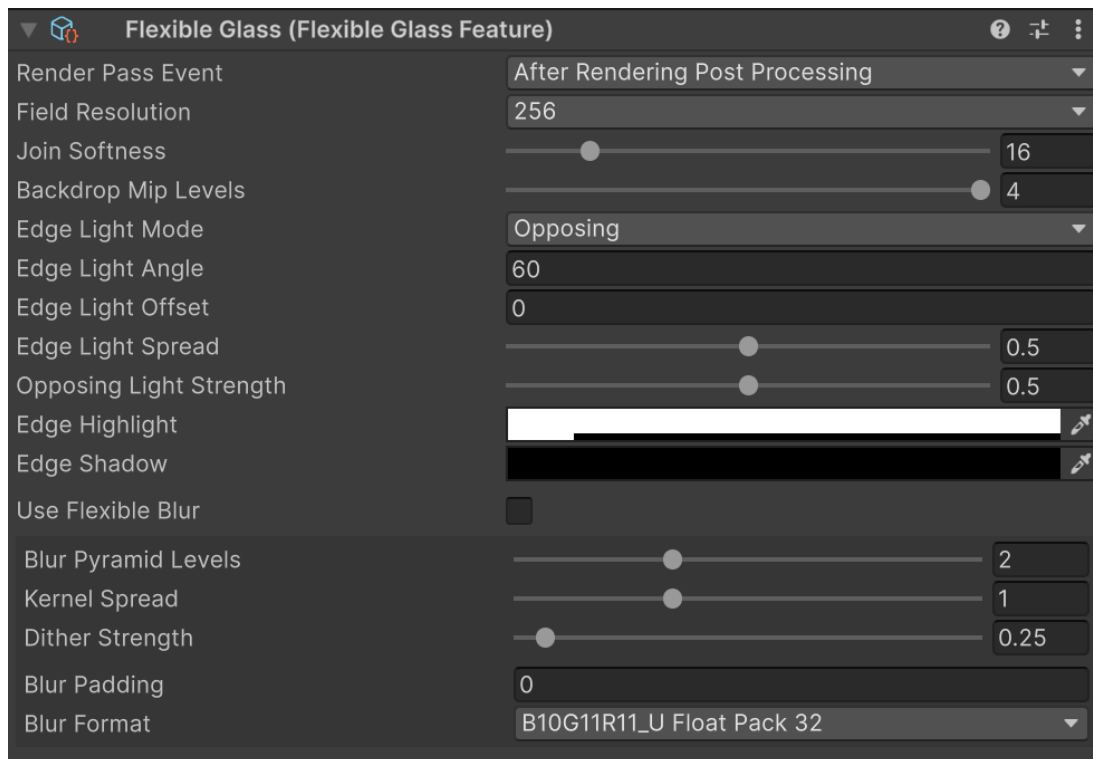


Figure 1. FlexibleGlassFeature on a Renderer Data asset (earlier Inspector). For Use Flexible Blur, see [Optional Flexible Blur integration](#). Lighting controls now use Lip rather than Edge; the Offset control has been removed.

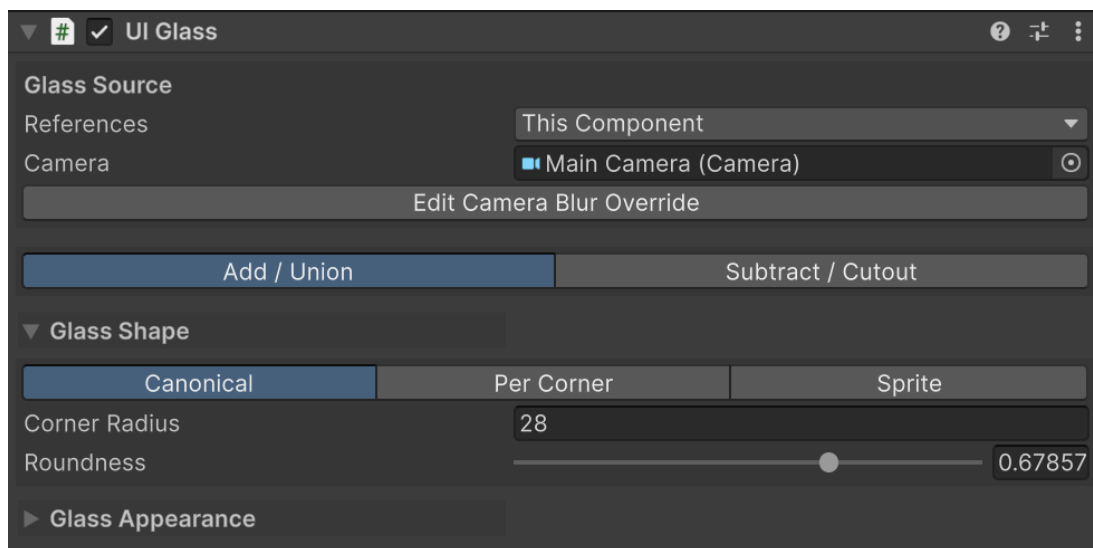


Figure 2. UIGlass source assignment and Canonical shape controls. Corner Radius sets the size; Roundness sets the curve.

Choosing a component

Both components support Canonical, Per Corner, and Sprite shapes, with the same physical refraction controls.

Requirement	UIGlass	GlassImage
Merge and subtract panels	Yes	No
Normal Canvas sibling ordering	No	Yes
Image fill and UI masking	No	Yes
Can be a Button's Target Graphic	No	Yes
Handles Graphic raycasts	No	Yes

UIGlass: joining and cutouts

UIGlass components referencing the same camera and Glass feature are combined. Add shapes join; Subtract shapes cut holes in the result. Their order in the hierarchy does not create stacked panes.

UIGlass is not a Graphic. It inherits Canvas Group opacity, but Mask and RectMask2D do not clip it. For pointer input, provide a separate Graphic. That Graphic determines the hit area; UIGlass does not shape it or clip child UI.

GlassImage: UGUI Image features

GlassImage derives from Image: it supports Canvas ordering, masking, Button transitions, and raycasting. Each image is separate; it cannot join UIGlass unions or cutouts.

Simple and Filled Image types are supported. Sliced and Tiled fall back to Simple. A fill boundary clips the glass and shadow; it does not create a new refractive edge.

GlassImages with the same source camera and feature number share that feature's capture and blur. The source camera can differ from the Canvas camera. Earlier Canvas siblings are not automatically included in the capture; see Cameras and layered UI for ordering.

Conversion

Component context menus convert between UIGlass and GlassImage, or Image and GlassImage. Glass settings are preserved, but features exclusive to the original component cannot transfer. Reassign scripts that referenced the replaced component.

Shapes

Both components have three shape modes. The RectTransform sets their local size.

Canonical

A rounded-rectangle model based on Apple's Liquid Glass, with two independent controls:

- **Corner Radius** sets the corner size in Canvas units, independently of the panel size. The radius is limited to half the shorter side. A radius of 0 produces square corners.
- **Roundness** changes the curve within those corners. At 1 the corners are circular; lower values make the transition into the straight edges flatter and more square-like. It does not change the requested Corner Radius.

Per Corner

The four fields are NW, NE, SW, and SE. Drag their labels to scrub values. The quick-adjust controls let you change corners together.

- **Chamfer:** the local Canvas-unit extent of each corner.
- **Normalize:** scales overlapping corner extents to fit the rectangle.
- **Squircle on:** exposes Smoothing. 0 is circular, 0.4 is a soft squircle, and 1 is squarer.
- **Squircle off:** exposes Concavity. 0 is rounded, 1 is a straight chamfer, and 2 is concave.

Use this mode when the corners need independent sizes or shapes.

Sprite

Assign Shape Sprite. Its alpha channel defines the outline. Lowering Alpha Threshold includes more translucent pixels, while raising it excludes them.

The contour is smoothed after applying the threshold.

Import as Sprite (2D and UI) with an alpha channel. Both components support rectangular and tightly packed Sprite Atlas entries, including rotations and flips. Packed sprites retain their original placement and transparent padding; the sprite mesh excludes neighbouring atlas entries. Custom mesh outlines also limit the sampled shape. Read/Write is not needed for field generation.

Shape examples



Figure 3. Canonical uses one corner radius and curve; Per Corner controls each corner independently; Sprite follows the alpha contour of the supplied image.

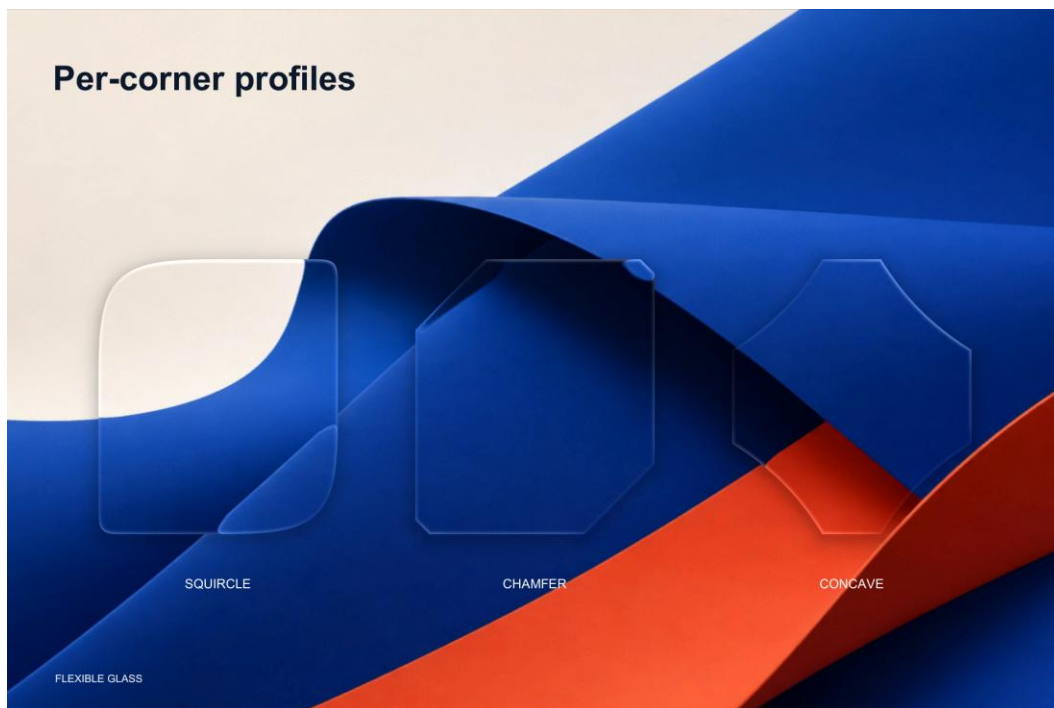


Figure 4. Per Corner with Squiracle enabled (left), Concavity 1 for straight chamfers (middle), and Concavity 2 for inward curves (right). The right panel uses larger corner extents.

Combining panels

UIGlass composition is grouped by camera and FlexibleGlassFeature number. GlassImage does not join that composition.

Add / Union

Overlapping Add shapes join into one glass outline. Their appearance settings blend through the join. Use matching settings when the result should look like one sheet of glass.

Subtract / Cutout

Subtract shapes cut the complete Add surface. They are not limited to the immediately preceding object in the hierarchy. A subtractor by itself does not draw a visible pane.

To cut only one group of panels, place that group and its subtractors on another FlexibleGlassFeature number. All participating shapes must reference the same camera and feature. A different feature is a different composition.

Join Softness

The feature's Join Softness rounds the joins and cutout intersections. It is measured in screen pixels. At 0, shapes meet at their exact outlines. Higher values soften their intersections and can join nearby shapes before their original outlines overlap.

Animation

Moving, rotating, or scaling the Transform reuses its field. Changing the effective local width, height, or corner settings requests the matching field from the cache; it is generated only if it is not already there. For a Preserve Aspect sprite, changing unused space in the RectTransform need not change the fitted shape size.

Union and subtraction examples

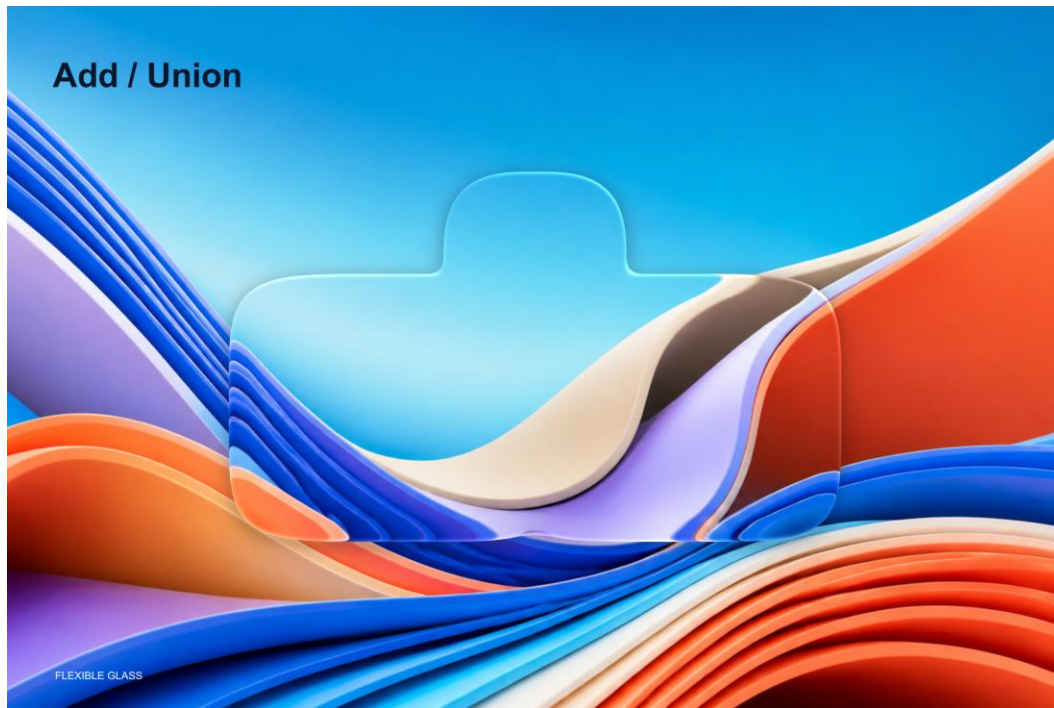


Figure 5. Three Add shapes joined into one surface. Join Softness rounds the junctions. All three reference the same camera and feature.

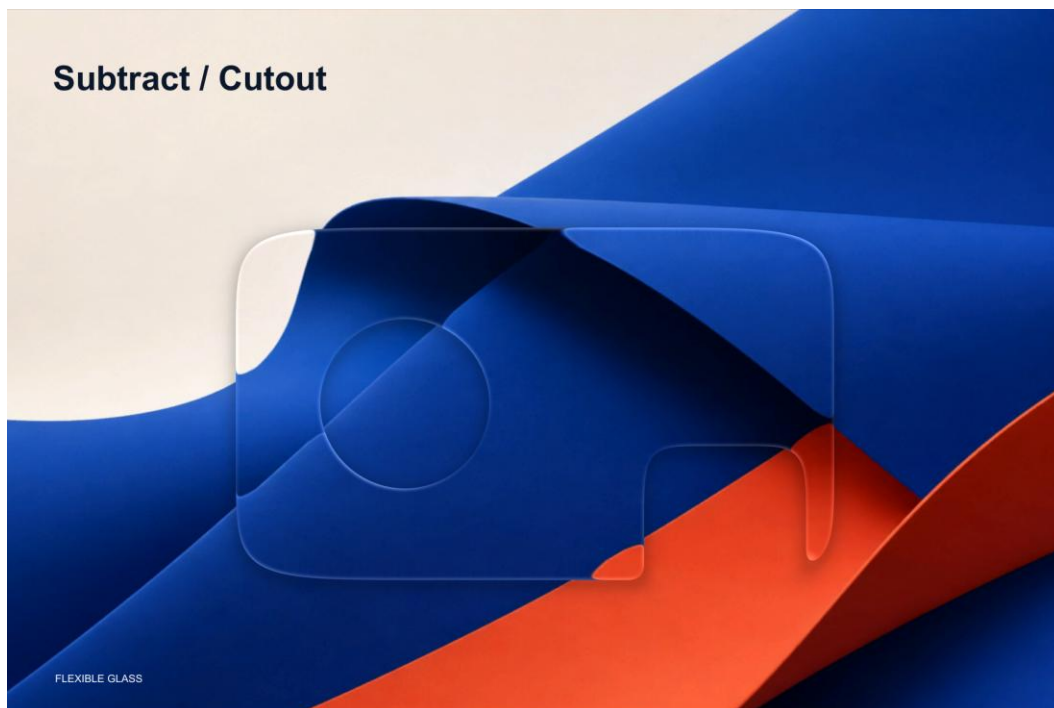


Figure 6. One Add panel with two Subtract shapes: a circular opening and a notch crossing the lower edge.

Surface and refraction

Color and transparency

Color supplies the glass RGB. **Color Mix** blends it into the captured background; at 0, the selected color has no effect. **Opacity** controls transparency and is multiplied by Canvas Group alpha. GlassImage uses Image.color for Color and Opacity, so Image fades and color transitions continue to work. UIGlass stores them in appearance.color.

Transmission scales background brightness before color and lip lighting are added. 1 leaves it unchanged; 0 removes the background contribution. Lowering Opacity instead blends the rendered glass with what is underneath, including the unwarped background. This can produce a double-image effect.

Optical shape

Optical Lip sets the width of the rounded edge that bends the background. **Optical Lip Units** selects Canvas units, percentage of the smaller side, or percentage of the larger side. Percentage mode labels the width **Optical Lip (%)**. The resolved width cannot exceed half the smaller side. At 0, the optical edge is flat.

Surface Smoothness ranges from 0.01 to 5. Higher values smooth a wider area without changing the outline; lip width is compensated. Canonical shapes offer **Custom** and **Auto**: Auto derives Smoothness from Optical Lip, Corner Radius, and Roundness. Per Corner and Sprite use the numeric value directly.

Depth Fallback, off by default on both components, uses geometric depth to reduce hard interior seams at high smoothness. It blends through UIGlass joins without regenerating fields.

Magnification zooms the captured background around the panel center without enlarging the panel. 1 is neutral.

Physical refraction

Lens Depth controls virtual travel through the glass relative to Optical Lip. 0 disables edge refraction. The slider covers 0-4 and the numeric field accepts up to 20. Large values can fold the background image over itself.

Refractive Index controls bending: 1 produces no refraction; the default is 1.5. **Abbe Number** controls dispersion: higher values give less color separation (default 64, range 0.1-64). Lens Depth and Refractive Index also affect the separation.

Lighting and shadows

The feature sets lip lighting; each component sets its band widths. Scene Lights do not control this lighting.

Feature lighting

- **Lip Light Mode: Directional** uses one shared screen-space direction, set by Lip Light Angle, with falloff centered on each element. The angle repeats every 360 degrees and may be animated through multiple turns.
- **Opposing**: adds a second light from the opposite direction. Opposing Light Strength sets its intensity relative to the first.
- **Point**: uses Point Light Position in normalized viewport coordinates. (0, 0) is the bottom-left and (1, 1) is the top-right. Values outside that range move the light off-screen. Point Light Radius is a fraction of viewport height, not object size.

Lip Light Spread sets the Directional/Opposing falloff width perpendicular to the light direction, relative to viewport height. Falloff follows each element, so moving it does not change its lighting. Centers blend at UIGlass joins. In Point mode, Spread sets angular coverage around the shared viewport light; moving an element changes its lighting. Spread 0 disables lip lighting.

Lip Highlight sets highlight color and alpha. **Lip Shadow** sets the complementary lip-shadow color and alpha; alpha 0 disables lip shadowing without disabling highlights.

Inner and outer lip lights

Lip Light Units selects **Percent of Optical Lip** or **Canvas Units (Absolute)** for both **Inner Lip Light** and **Outer Lip Light**. On a 20-unit Optical Lip, 10% and 2 Canvas units give the same band width before soft falloff. Both widths are capped at the Optical Lip; 0 removes the band.

Each light highlights an outer lip and the opposing inner lip; Opposing mode adds the reverse pairing. Lip Shadow colors the less-lit areas. Increasing Optical Lip widens percentage-based bands; absolute widths stay fixed unless capped. Spread controls reach around the perimeter, not band width. Lens Depth does not widen the bands.

Drop shadow

Shadow Color, **Shadow Size**, and **Shadow Offset** belong to each component. Size and offset use screen pixels. Color alpha controls opacity; white or colored shadows can be used as glows. The shadow is separate from Lip Shadow.

Alpha 0 disables the shadow, as does Size 0 with Offset (0, 0). Dithering reduces banding automatically.

Blur and renderer settings

Built-in blur

Blur Pyramid Levels ranges from 0 to 6. 0 bypasses the blur; higher values downsample and reconstruct the captured background for a wider blur. **Kernel Spread** adjusts tap spacing and is normalized to a 1080-pixel render height.

Dither Strength reduces blur banding with interleaved-gradient noise. This is independent of drop-shadow dithering.

Blur Padding adds capture space beyond the automatic optical/filter bounds, without enlarging the glass or shadow. **Blur Format** selects the texture format for built-in blur.

Backdrop Mip Levels

Backdrop mips filter the sampled image where refraction compresses it, even when blur is off. Choose 0 for the highest-resolution backdrop only, or add up to 8 lower levels (default 4). With blur enabled, these mips come from the blurred result. This setting is separate from Field Resolution and its optical-field mips.

Field Resolution

Choose 64, 128, 256, 512, or 1024 for retained optical fields. 256 is the default. Lower values save field memory and generation work but may make the optical interior less smooth. Doubling the resolution quadruples the texel count.

Optional Flexible Blur integration

The **Flexible UI Suite** is pre-integrated. With the standalone assets, import `FlexibleGlassIntegration.unitypackage` from `Assets/Plugins/FlexibleUI/FlexibleBlur` after importing both Flexible Glass and Flexible Blur. That integration package is supplied with standalone Flexible Blur.

Add `FlexibleBlurFeature` to the source camera's renderer. On `FlexibleGlassFeature`, enable **Use Flexible Blur**, select the Blur feature number, and choose a Blur Preset or local Blur Settings.

`FlexibleBlurFeature` must run before `FlexibleGlassFeature`: use an earlier Render Pass Event, or place it earlier in the renderer list when both use the same event. Built-in Kernel Spread, Dither Strength, and Blur Format do not configure Flexible Blur.

Clear and frosted examples

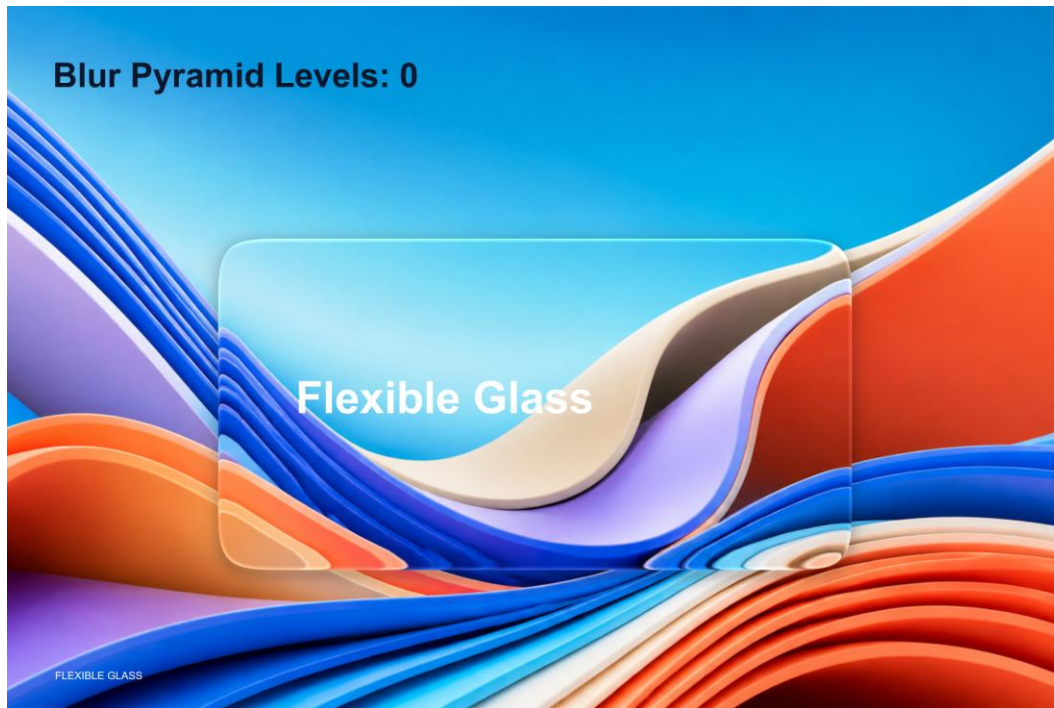


Figure 7. Blur Pyramid Levels 0. The background is sharp apart from the refracted edge.

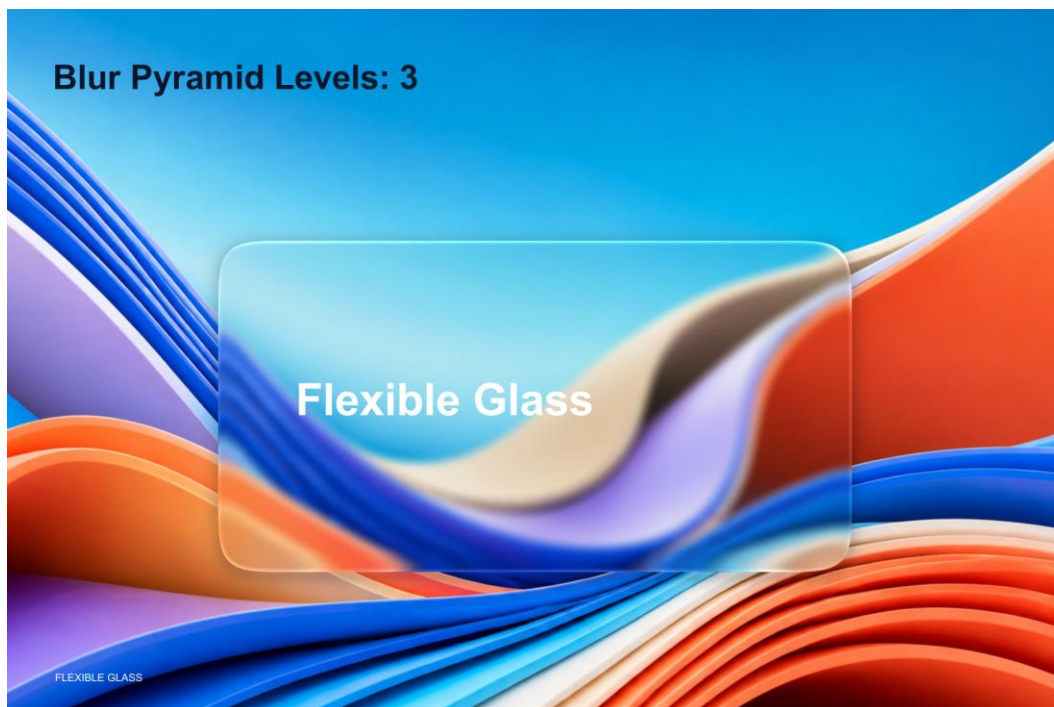


Figure 8. The same panel with Blur Pyramid Levels 3.

Cameras and layered UI

What is captured, and what is drawn

The Camera in **Glass Source** selects the camera image to capture. FlexibleGlassFeature captures that image at its **Render Pass Event**, prepares the blur, and then draws the UIGlass components assigned to it. GlassImage samples the prepared capture when its Canvas draws.

The capture contains only what has already been drawn. It does not include that feature's own UIGlass draw, later camera passes, or Screen Space - Overlay UI. A later Glass feature or camera can capture those earlier results.

One camera with Overlay UI

For glass over a 3D scene, use the camera as Glass Source and keep the feature at **After Rendering Post Processing**. Screen Space - Overlay Canvas graphics draw after the camera. A GlassImage on that Canvas appears in normal sibling order; UIGlass appears beneath those graphics because the feature drew it earlier.

An Image behind GlassImage on the same Overlay Canvas is still too late to be captured. To use UI as the background, draw it on a Screen Space - Camera Canvas assigned to the source camera, then draw the foreground glass on the Overlay Canvas.

Camera-rendered glass

For a GlassImage drawn in the normal transparent pass by a Screen Space - Camera or World Space Canvas on its own source camera, use **Before Rendering Transparents**. The feature captures the opaque scene and skybox before the Canvas draws. Camera-rendered UI and other transparent objects drawn later are not in that capture.

Use **After Rendering Transparents** for UIGlass that must appear over camera-rendered UI and transparent objects. A GlassImage drawn in that same transparent pass cannot use this later capture in the current frame. To draw it after the capture on the same camera, use Render Objects as described next.

Before Rendering Post Processing lets post-processing affect the glass drawn by the feature. **After Rendering Post Processing** places that draw after the main post-processing pass. Final antialiasing and output conversion can still run afterward. See Unity's [RenderPassEvent reference](#) for the URP event definitions.

Single-camera stacked blur

The back glass blurs the scene. The front glass blurs the scene **and the back UI**, including its text and controls. To do this with one camera, capture the background, draw the back Canvas, capture that result, then draw the front Canvas.

[Watch the Flexible Blur single-camera stacking demonstration \(10:11\)](#). The principle is the same: use Glass features, GlassReferenceProviders, and UIGlass or GlassImage instead of their Flexible Blur counterparts. Flexible Glass's built-in blur supports this setup.

Set up the Canvases

1. Create two **GameObject layers** in Tags and Layers: **UI Back** and **UI Front**. These are example names for layers you create, not built-in layers.
2. Create **Back Canvas** and **Front Canvas**, both **Screen Space - Camera**, assigned to the same camera. Put Back Canvas and all its children on UI Back; put Front Canvas and all its children on UI Front.
3. Add a **GlassReferenceProvider** to each Canvas. Assign the shared Camera on both. Set **Feature Number to 0 on Back Canvas, and 1 on Front Canvas**. On their glass components, choose **References > Canvas Provider**.
4. Leave both layers checked in the camera's **Culling Mask**. Uncheck them in the Renderer Data's **Filtering > Transparent Layer Mask**. This prevents the normal transparent pass from drawing these Canvases; Render Objects will draw them at the stages below.

Set up the renderer

Add these four features to that camera's Renderer Data, in this exact order. Set all four Render Pass Events to **After Rendering Post Processing**:

1. **FlexibleGlassFeature #0** captures the scene for the back glass.
2. **Render Objects - Back**, with Layer Mask **UI Back**, draws Back Canvas.
3. **FlexibleGlassFeature #1** captures that result, including Back Canvas's controls and text.
4. **Render Objects - Front**, with Layer Mask **UI Front**, draws Front Canvas using the new capture.

On both Render Objects features, use **Queue: Transparent** and leave material and shader overrides off. To ignore scene depth, enable the Depth override, select **Always**, and disable depth writes. If post-processing should affect the UI, use **After Rendering Transparents** on all four features instead.

GlassImage draws with its Canvas. UIGlass draws in its assigned Glass feature, before that Canvas's graphics. To add another level, repeat the Canvas/layer/provider setup with Feature Number 2 and append another Glass / Render Objects pair.

Two features on one camera

Use two Glass features when you need separate captures or UIGlass groups at different stages. For example, a lower glass panel can sit beneath camera-rendered controls while an upper panel refracts those controls:

1. Feature #0 runs **Before Rendering Transparents**. It captures the scene and draws the lower UIGlass group.
2. The camera draws transparent objects and camera-rendered UI above that group.
3. Feature #1 runs **After Rendering Transparents**. It captures the result and draws the upper UIGlass group.
4. Screen Space - Overlay UI draws last. GlassImages there may select either capture by Feature #.

Assign each UIGlass group to its intended feature number. This also separates unions and cutouts. Feature # identifies the Glass feature's position among Glass features on the renderer; Render Pass Event determines its execution stage.

Different events let the normal transparent pass run between these two captures. For more UI levels, use the explicit Render Objects sequence on the preceding page; putting several Glass features together without intervening UI draws does not produce that sequence.

Stacking cameras for windows

A camera stack is another way to blur or refract lower windows and their contents. A URP **Overlay camera** adds to the Base camera's image; it is not a **Screen Space - Overlay Canvas**.

1. The Base camera draws the background. Its Glass feature runs **After Rendering Transparents**, capturing that camera's scene and UI.
2. The first Overlay camera draws the lower window's Screen Space - Camera Canvas. Its GlassImages reference the Base camera's capture.
3. A Glass feature on that Overlay camera runs **After Rendering Transparents**, capturing the completed lower window.
4. The next Overlay camera draws the upper window. Its GlassImages reference the preceding camera's capture.

Use separate culling layers so each Canvas is drawn only by its assigned camera, and put the Overlay cameras in back-to-front stack order. Each source camera needs the referenced Glass feature on its renderer. See Unity's [camera stacking documentation](#) for stack setup.

Sharing source assignments

Add **GlassReferenceProvider** to a Canvas and choose its Camera and Feature Number. On glass components using that Canvas, set References to **Canvas Provider**. Add a provider to nested Canvases too: each component looks on its nearest parent Canvas.

Camera-specific overrides

Add **FlexibleGlassCameraOverride** to a camera and select the FlexibleGlassFeature with **Feature #**. Composition, refraction, lighting, and blur overrides are enabled independently. Render Pass Event and Field Resolution remain on the feature because the render pass and retained field cache are shared.

The override applies in Edit mode and Play mode. Disable a group to inherit that group from the feature, or disable the component to inherit everything. For GlassImage, put the override on its source camera, which may not be the Canvas camera.

VR

For glass that scene objects must occlude, use GlassImage on a World Space Canvas. UIGlass composites over the camera image without scene-depth testing, even when its RectTransform is in world space.

Raycasting and scripting

GlassImage raycasting

The **Raycast Target** selector has three modes:

- **Disabled:** the GlassImage does not receive Graphic raycasts.
- **Standard:** procedural shapes use their geometry. Sprite shapes use the fitted rectangle, or `Image.alphaHitTestMinimumThreshold` if configured.
- **Advanced:** tests the cached distance field for any shape mode, including packed sprites. It does not require Read/Write on the source texture.

In Advanced mode, **Raycast Padding** adjusts the hit area in Canvas units. Positive padding shrinks it; negative padding expands it, matching normal Image raycast padding. Zero follows the visible shape boundary. The adjustment does not change the rendered glass or regenerate the field. Image fill and parent masks still clip hits.

Advanced mode needs asynchronous GPU readback. A newly generated shape accepts hits once its CPU field is ready. Use Standard on devices without this capability.

Updating values

The public types are in **JeffGrawAssets.FlexibleUI**. GlassImage's appearance and shape properties update the image through their setters. For an existing GlassImage named `image`:

```
image.LensDepth = 2f;  
image.AbbeNumber = 32f;  
image.OpticalLip = 14f;  
image.CanonicalCornerRadius = 28f;  
image.DepthFallback = true; // Optional interior-seam correction.
```

For runtime blur changes, enable **FlexibleGlassCameraOverride.OverrideBlur** and set **Iterations** on the source camera.

Performance and retained fields

What is cached

Descriptors include local size, padding, procedural settings, sprite, texture/region and update count, and Alpha Threshold. Matching descriptors share a field. Position, rotation, Optical Lip, Surface Smoothness, Depth Fallback, color, refraction, lip lighting, and shadow do not change it.

On a cache miss, the GPU generates the field and its mips. Sprite normals are automatically smoothed to reduce low-Smoothness refraction noise, leaving distance channels and outlines unchanged. Procedural shapes skip this step. Later frames reuse the result; Transform scaling does not rebuild it.

Advanced raycasting shares asynchronous CPU field copies between matching images. Raycast Padding reuses them.

Capacity and reuse

Capacity grows to a power of two to hold unique live and protected sprite descriptors. Five identical components occupy one slice. Capacity does not shrink.

Fields needed by the current render cannot be evicted. A cache miss uses a free slot, then the least recently used eligible slot. Unused procedural fields can be replaced. Each sprite's most recently requested variant has extra eviction protection. A new variant transfers that protection; changing the active scene clears it. Neither immediately deletes the old field.

Each renderer pass owns a cache. Ordinary feature-setting changes reuse a valid pass and its fields. Changing Field Resolution rebuilds field resources; disposing or replacing the pass releases its cache. Camera blur overrides do not change shape descriptors.

Where to spend the budget

- **Coverage and overlap:** larger regions shade more pixels. UIGlass cost also grows with the number of participating shapes.
- **Cameras:** cap stacking or share a backdrop. Each camera has overhead beyond its blur.
- **Dispersion:** color separation adds samples. Use it where visible.
- **Shadows:** wide shadows expand coverage. Alpha 0, or size 0 with zero offset, disables them.
- **Generation:** changing local size, sprite content, or corners can miss the cache. Translation and rotation reuse fields.

Profile a target-device player build. Check the optical interior when lowering Field Resolution, and measure first-use field generation separately from steady-state rendering.

Troubleshooting

Nothing renders: check URP, compute/shader model 4.5 support, the source camera and feature, parent Canvas, opacity, and Sprite assignment. Read the source diagnostics and Console warnings.

Wrong background: check the source assignment and capture order. A capture cannot include later Canvas graphics. For camera-rendered GlassImages, capture before their Canvas draw or use a capture from a preceding camera.

Blur controls have no effect: check camera overrides, Use Flexible Blur, and the active renderer/Quality asset. Blur Pyramid Levels 0 bypasses blur. Backdrop mips are not a blur radius. If Use Flexible Blur is enabled, Flexible Blur must be installed and render before the glass pass.

Smearing or darkening: low Opacity blends warped and unwarped images. Color Mix and Transmission affect brightness; Lens Depth, Refractive Index, and Magnification affect distortion. If enabling post-processing changes the whole screen, check the active Volume's tonemapping, exposure, and color adjustments.

Hard interior seam: try Depth Fallback for seams at high Surface Smoothness.

Mock HMD with single-pass MSAA: one-eye color artifacts occurred in Unity 6.0/6.5 even without Flexible Glass. Setting camera Background Color to black avoided them; a skybox can still be used. This Editor workaround has not been verified on a headset.

Report issues to hurleybird@gmail.com with the Unity/URP version, graphics API, device, camera setup, and a small reproduction.